

Binary Tree In-Order Traversal (Left -> Root -> Right)

1. Recursive

```
public void inorder(TreeNode root, List<Integer> res) {  
    if (root == null) return;  
    inorder(root.left, res);  
    res.add(root.val);  
    inorder(root.right, res);  
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$ (Call stack, worst $O(N)$, avg $O(\log N)$)
-

2. Iterative (Stack)

```
public List<Integer> inorderIterative(TreeNode root) {  
    List<Integer> res = new ArrayList<>();  
    Stack<TreeNode> stack = new Stack<>();  
    TreeNode curr = root;  
  
    while (curr != null || !stack.isEmpty()) {  
        while (curr != null) { // Go to leftmost node  
            stack.push(curr);  
            curr = curr.left;  
        }  
        curr = stack.pop();  
        res.add(curr.val);    // Visit root  
        curr = curr.right;    // Go to right subtree  
    }  
    return res;  
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$ (Explicit stack)
-

3. Morris Traversal ($O(1)$ Space)

Idea: Thread the right child of the inorder predecessor back to `curr`.

```

public List<Integer> morrisInorder(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    TreeNode curr = root;

    while (curr != null) {
        if (curr.left == null) {
            res.add(curr.val);    // No left child, visit and go right
            curr = curr.right;
        } else {
            TreeNode pred = curr.left;
            // Find inorder predecessor
            while (pred.right != null && pred.right != curr) {
                pred = pred.right;
            }
            if (pred.right == null) { // Threading: Connect pred to curr
                pred.right = curr;
                curr = curr.left;
            } else {                // Unthreading: Restore tree
                pred.right = null;
                res.add(curr.val);    // Visit
                curr = curr.right;
            }
        }
    }
    return res;
}

```

- **Complexity:** Time $O(N)$ (Each edge visited at most twice), Space $O(1)$

Binary Tree Pre-Order Traversal (Root -> Left -> Right)

1. Recursive

```

public void preorder(TreeNode root, List<Integer> res) {
    if (root == null) return;
    res.add(root.val);    // Visit root
    preorder(root.left, res);
    preorder(root.right, res);
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

2. Iterative (Stack)

Idea: Push right child first, then left child, so left is popped first from stack.

```

public List<Integer> preorderIterative(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Stack<TreeNode> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode curr = stack.pop();
        res.add(curr.val);           // Visit root

        if (curr.right != null) stack.push(curr.right);
        if (curr.left != null) stack.push(curr.left);
    }
    return res;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

3. Morris Traversal ($O(1)$ Space)

Idea: Visit `curr` node before descending into its left subtree during threading phase.

```

public List<Integer> morrisPreorder(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    TreeNode curr = root;

    while (curr != null) {
        if (curr.left == null) {
            res.add(curr.val);    // No left child, visit and go right
            curr = curr.right;
        } else {
            TreeNode pred = curr.left;
            // Find inorder predecessor
            while (pred.right != null && pred.right != curr) {
                pred = pred.right;
            }
            if (pred.right == null) { // Threading
                res.add(curr.val);    // Visit root BEFORE going left
                pred.right = curr;
                curr = curr.left;
            } else {                // Unthreading: Restore tree
                pred.right = null;
                curr = curr.right;
            }
        }
    }
    return res;
}

```

- **Complexity:** Time $O(N)$, Space $O(1)$

Binary Tree Post-Order Traversal (Left -> Right -> Root)

1. Recursive

```

public void postorder(TreeNode root, List<Integer> res) {
    if (root == null) return;
    postorder(root.left, res);
    postorder(root.right, res);
    res.add(root.val);          // Visit root
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

2. Iterative (1 Stack + Reverse)

Idea: Generate Root -> Right -> Left sequence using a stack, then reverse the entire list.

```

public List<Integer> postorderIterative(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Stack<TreeNode> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode curr = stack.pop();
        res.add(curr.val);          // Root -> Right -> Left

        // Push LEFT first so RIGHT is popped first
        if (curr.left != null) stack.push(curr.left);
        if (curr.right != null) stack.push(curr.right);
    }
    Collections.reverse(res);      // Left -> Right -> Root
    return res;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

2b. Iterative (True 1 Stack + `prev` Pointer)

Idea: Go as far left as possible. Only visit a node if it has no right child or its right child was just visited. Otherwise, peek and explore right.

```

public List<Integer> postorderIterativeTrue(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Stack<TreeNode> stack = new Stack<>();
    TreeNode curr = root;
    TreeNode prev = null;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }

        curr = stack.peek(); // Don't pop yet!

        // If no right child OR right child was the last visited node
        if (curr.right == null || curr.right == prev) {
            stack.pop();
            res.add(curr.val); // Visit
            prev = curr;      // Mark as visited
            curr = null;      // Force stack.peek() in next iteration
        } else {
            curr = curr.right; // Process right subtree first
        }
    }
    return res;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

3. Morris Traversal ($O(1)$ Space)

Idea: Use a dummy root node, and reverse the right spine of the predecessor during unthreading.

```

public List<Integer> morrisPostorder(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    TreeNode dummy = new TreeNode(0);
    dummy.left = root;
    TreeNode curr = dummy;

    while (curr != null) {
        if (curr.left == null) {
            curr = curr.right;
        } else {
            TreeNode pred = curr.left;
            // Find inorder predecessor
            while (pred.right != null && pred.right != curr) pred = pred.right;

            if (pred.right == null) { // Threading
                pred.right = curr;
                curr = curr.left;
            } else { // Unthreading
                addReversePath(curr.left, pred, res);
                pred.right = null;
                curr = curr.right;
            }
        }
    }
    return res;
}

// Helper to extract values in reverse order along the right spine
private void addReversePath(TreeNode from, TreeNode to, List<Integer> res) {
    List<Integer> temp = new ArrayList<>();
    TreeNode curr = from;
    while (true) {
        temp.add(curr.val);
        if (curr == to) break;
        curr = curr.right;
    }
    Collections.reverse(temp);
    res.addAll(temp);
}

```

- **Complexity:** Time $O(N)$, Space $O(1)$ (Pointer-revo is $O(1)$, arraylist reverse is $O(h)$ depending on spine length but often allowed/accepted as an easy explanation).

Binary Tree Level-Order Traversal (BFS)

1. Top to Bottom

Idea: Use a Queue to process nodes level by level. Track `size` of queue to know level boundaries.

```

public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> currentLevel = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            currentLevel.add(curr.val);

            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
        res.add(currentLevel);
    }
    return res;
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$ (Queue can hold at most $N/2$ nodes at the leaf level)

2. Bottom to Up

Idea: Same as Top-Bottom BFS, but prepend each new level using `res.add(0, currentLevel)` (use `LinkedList` for $O(1)$ prepend).

```

public List<List<Integer>> levelOrderBottom(TreeNode root) {
    List<List<Integer>> res = new LinkedList<>(); // LinkedList for  $O(1)$  prepend
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> currentLevel = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            currentLevel.add(curr.val);

            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
        res.add(0, currentLevel); // Prepend level!
    }
    return res;
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

Decision Tree: Which Traversal to Use?

1. Do you need to visit nodes level by level? (e.g., Level averages, Right side view) ☑ YES: Use BFS (Queue).
2. Do you need to evaluate deepest nodes first to compute something for its parents? (e.g., Heights, Subtree sums, Lowest Common Ancestor) ☑ YES: Use DFS / Post-Order (Recursion or Stack).
3. Is it a Binary Search Tree (BST) and you need sorted order? (e.g., K-th smallest, Validate BST) ☑ YES: Use DFS / In-Order (Recursion or Stack).
4. Are you processing the exact structure top-down? (e.g., Serialization, Copying a tree, Path Sum) ☑ YES: Use DFS / Pre-Order.
5. Is the interviewer explicitly asking for $O(1)$ extra space (and it's NOT a level-order problem)? ☑ YES: Use Morris Traversal (temporarily threads the tree).

Tree Patterns: BFS Variations

199. Binary Tree Right Side View

*Idea: BFS, add the **last** node of each level to result.*

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            if (i == size - 1) res.add(curr.val); // Last node in level
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
    }
    return res;
}
```

513. Find Bottom Left Tree Value

*Idea: BFS from **Right to Left**. The last visited node is the bottom-left.*

```
public int findBottomLeftValue(TreeNode root) {
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    TreeNode curr = null;
    while (!q.isEmpty()) {
        curr = q.poll();
        if (curr.right != null) q.offer(curr.right); // RIGHT first
        if (curr.left != null) q.offer(curr.left);  // LEFT second
    }
    return curr.val; // Last node processed is bottom-left
}
```

637. Average of Levels in Binary Tree

Idea: Standard BFS, compute sum for the level and divide by size.


```

public List<Double> averageOfLevels(TreeNode root) {
    List<Double> res = new ArrayList<>();
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        double sum = 0;
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            sum += curr.val;
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
        res.add(sum / size);
    }
    return res;
}

```

515. Find Largest Value in Each Tree Row

Idea: BFS, track max per level.

```

public List<Integer> largestValues(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        int max = Integer.MIN_VALUE;
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            max = Math.max(max, curr.val);
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
        res.add(max);
    }
    return res;
}

```

2415. Reverse Odd Levels of Binary Tree

Idea: Perform DFS symmetrically on two nodes, swapping their values if level is odd.

```

public TreeNode reverseOddLevels(TreeNode root) {
    dfs(root.left, root.right, 1);
    return root;
}
private void dfs(TreeNode left, TreeNode right, int level) {
    if (left == null || right == null) return;
    if (level % 2 == 1) { // Odd level: swap values
        int temp = left.val;
        left.val = right.val;
        right.val = temp;
    }
    // Symmetric traversal: outer pairs, inner pairs
    dfs(left.left, right.right, level + 1);
    dfs(left.right, right.left, level + 1);
}

```

Tree Patterns: Depth & Properties

104. Maximum Depth of Binary Tree

Idea: Recursively find $\max(\text{left}, \text{right}) + 1$.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

111. Minimum Depth of Binary Tree

Idea: Recursively find $\min(\text{left}, \text{right}) + 1$. Handle skew trees: if one child is null, we MUST take the other child path.

```
public int minDepth(TreeNode root) {
    if (root == null) return 0;
    if (root.left == null) return 1 + minDepth(root.right);
    if (root.right == null) return 1 + minDepth(root.left);
    return 1 + Math.min(minDepth(root.left), minDepth(root.right));
}
```

110. Balanced Binary Tree

Idea: Bottom-up DFS. Return heights, but if a subtree is unbalanced, return -1 to fast-fail.

```
public boolean isBalanced(TreeNode root) {
    return dfsHeight(root) != -1;
}

private int dfsHeight(TreeNode root) {
    if (root == null) return 0;

    int leftHeight = dfsHeight(root.left);
    if (leftHeight == -1) return -1;

    int rightHeight = dfsHeight(root.right);
    if (rightHeight == -1) return -1;

    if (Math.abs(leftHeight - rightHeight) > 1) return -1;
    return Math.max(leftHeight, rightHeight) + 1;
}
```

100. Same Tree

Idea: Base cases first (both null, or one null), then compare values and recurse on left & right.

```
public boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

101. Symmetric Tree

Idea: Similar to Same Tree, but we compare left.left with right.right and left.right with right.left .

```
public boolean isSymmetric(TreeNode root) {
    return root == null || isMirror(root.left, root.right);
}
private boolean isMirror(TreeNode t1, TreeNode t2) {
    if (t1 == null && t2 == null) return true;
    if (t1 == null || t2 == null || t1.val != t2.val) return false;
    return isMirror(t1.left, t2.right) && isMirror(t1.right, t2.left);
}
```

Tree Patterns: Modifications & Misc

226. Invert Binary Tree

Idea: Swap `left` and `right` children recursively.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(temp);
    return root;
}
```

617. Merge Two Binary Trees

Idea: Add overlapping node values. If one node is null, return the other. Recursively merge children.

```
public TreeNode mergeTrees(TreeNode root1, TreeNode root2) {
    if (root1 == null) return root2;
    if (root2 == null) return root1;

    TreeNode merged = new TreeNode(root1.val + root2.val);
    merged.left = mergeTrees(root1.left, root2.left);
    merged.right = mergeTrees(root1.right, root2.right);
    return merged;
}
```

655. Print Binary Tree

Idea: Get `height`, create a `height x (2^height - 1)` matrix of `""`. Recursively populate `res[row][col]` where root is at `col = (left + right) / 2`.

```

public List<List<String>> printTree(TreeNode root) {
    int h = getHeight(root);
    int m = h, n = (1 << h) - 1; // 2^h - 1
    List<List<String>> res = new ArrayList<>();
    for (int i = 0; i < m; i++) {
        res.add(new ArrayList<>(Collections.nCopies(n, "")));
    }
    fill(root, res, 0, 0, n - 1);
    return res;
}

private int getHeight(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(getHeight(root.left), getHeight(root.right));
}

private void fill(TreeNode root, List<List<String>> res, int r, int l, int rIdx) {
    if (root == null) return;
    int mid = l + (rIdx - l) / 2;
    res.get(r).set(mid, String.valueOf(root.val));
    fill(root.left, res, r + 1, l, mid - 1);
    fill(root.right, res, r + 1, mid + 1, rIdx);
}

```

N-ary Tree Patterns

Node Structure:

```

class Node {
    public int val;
    public List<Node> children;
}

```

589/590. Preorder & Postorder (Recursive)

Idea: Iterate over children list instead of just left and right.

```

public void preorder(Node root, List<Integer> res) {
    if (root == null) return;
    res.add(root.val);           // Root
    for (Node sub : root.children) // Left to Right
        preorder(sub, res);
}

public void postorder(Node root, List<Integer> res) {
    if (root == null) return;
    for (Node sub : root.children) // Left to Right
        postorder(sub, res);
    res.add(root.val);           // Root
}

```

559. Maximum Depth of N-ary Tree

Idea: Default depth is 1. Max depth over all children + 1.

```
public int maxDepth(Node root) {
    if (root == null) return 0;
    int max = 0;
    for (Node sub : root.children) {
        max = Math.max(max, maxDepth(sub));
    }
    return max + 1;
}
```

Phase 2: Week 2 — BST Fundamentals & Operations

BST Properties: Left subtree values < Root < Right subtree values. In-order traversal of a BST yields a **sorted** array.

1. Core BST Properties & Traversals

98. Validate Binary Search Tree

Idea: Pass down valid boundaries (`min`, `max`). Use `Long` to avoid integer overflow edge cases.

```
public boolean isValidBST(TreeNode root) {
    return isValid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}
private boolean isValid(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return isValid(node.left, min, node.val) && isValid(node.right, node.val, max);
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

235. Lowest Common Ancestor of a BST

Idea: Leverage the BST property. If both `p` and `q` are smaller, LCA is in left subtree. If both larger, LCA is in right subtree. Otherwise, we've split or found one of them (so current node is LCA).

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) return null;
    if (p.val < root.val && q.val < root.val) return lowestCommonAncestor(root.left, p, q);
    if (p.val > root.val && q.val > root.val) return lowestCommonAncestor(root.right, p, q);
    return root; // Split occurs here
}
```

- **Complexity:** Time $O(H)$, Space $O(H)$ (recursion stack)

230. Kth Smallest Element in a BST

Idea: In-order traversal visits nodes in ascending order. Keep a count and return when `count == k`.

```

int count = 0;
int result = -1;
public int kthSmallest(TreeNode root, int k) {
    inorder(root, k);
    return result;
}
private void inorder(TreeNode node, int k) {
    if (node == null || result != -1) return;
    inorder(node.left, k);
    count++;
    if (count == k) {
        result = node.val;
        return;
    }
    inorder(node.right, k);
}

```

- **Complexity:** Time $O(H + k)$, Space $O(H)$

538 / 1038. Convert BST to Greater Tree

Idea: Reverse In-Order traversal (Right -> Root -> Left). Maintain a running sum and update node values.

```

int sum = 0;
public TreeNode bstToGst(TreeNode root) {
    if (root != null) {
        bstToGst(root.right);
        sum += root.val;
        root.val = sum;
        bstToGst(root.left);
    }
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

530 / 783. Minimum Absolute Difference in BST

Idea: In-order traversal yields sorted elements. The minimum difference must be between two adjacent elements in the sorted order. Keep track of the `prev` node.

```

Integer prev = null;
int minDiff = Integer.MAX_VALUE;
public int getMinimumDifference(TreeNode root) {
    if (root == null) return minDiff;
    getMinimumDifference(root.left);
    if (prev != null) {
        minDiff = Math.min(minDiff, root.val - prev);
    }
    prev = root.val;
    getMinimumDifference(root.right);
    return minDiff;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

653. Two Sum IV - Input is a BST

Idea: DFS + HashSet. Or Inorder to list and use 2-pointers.

```

public boolean findTarget(TreeNode root, int k) {
    Set<Integer> set = new HashSet<>();
    return dfs(root, set, k);
}

private boolean dfs(TreeNode root, Set<Integer> set, int k) {
    if (root == null) return false;
    if (set.contains(k - root.val)) return true;
    set.add(root.val);
    return dfs(root.left, set, k) || dfs(root.right, set, k);
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

938. Range Sum of BST

Idea: DFS. Optimization: only traverse left if `root.val > low`, only traverse right if `root.val < high`.

```

public int rangeSumBST(TreeNode root, int low, int high) {
    if (root == null) return 0;
    int sum = 0;
    if (root.val >= low && root.val <= high) sum += root.val;
    if (root.val > low) sum += rangeSumBST(root.left, low, high);
    if (root.val < high) sum += rangeSumBST(root.right, low, high);
    return sum;
}

```

- **Complexity:** Time $O(N)$ (worst case), Space $O(H)$

2. Modifying BSTs & Construction

701. Insert into a Binary Search Tree

Idea: Traverse until you find a `null` spot, then attach the new node.

```

public TreeNode insertIntoBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    if (val < root.val) root.left = insertIntoBST(root.left, val);
    else root.right = insertIntoBST(root.right, val);
    return root;
}

```

- **Complexity:** Time $O(H)$, Space $O(H)$

450. Delete Node in a BST

Idea: Three cases: Node is leaf (return null), Node has 1 child (return the child), Node has 2 children (find inorder successor, replace value, delete successor in right subtree).

```

public TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null;
    if (key < root.val) root.left = deleteNode(root.left, key);
    else if (key > root.val) root.right = deleteNode(root.right, key);
    else {
        // Found the node
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        // Node with two children: Get the inorder successor (smallest in the right subtree)
        TreeNode minNode = getMin(root.right);
        root.val = minNode.val;
        // Delete the inorder successor
        root.right = deleteNode(root.right, minNode.val);
    }
    return root;
}

private TreeNode getMin(TreeNode node) {
    while (node.left != null) node = node.left;
    return node;
}

```

- **Complexity:** Time $O(H)$, Space $O(H)$

1008. Construct Binary Search Tree from Preorder Traversal

Idea: Use an upper bound. The left child must be strictly less than root.

```

int i = 0;
public TreeNode bstFromPreorder(int[] preorder) {
    return build(preorder, Integer.MAX_VALUE);
}

private TreeNode build(int[] preorder, int bound) {
    if (i == preorder.length || preorder[i] > bound) return null;
    TreeNode root = new TreeNode(preorder[i++]);
    root.left = build(preorder, root.val);
    root.right = build(preorder, bound);
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

108. Convert Sorted Array to Binary Search Tree

Idea: Choose the middle element as root to ensure height-balance. Recursively build left and right subtrees.

```

public TreeNode sortedArrayToBST(int[] nums) {
    return build(nums, 0, nums.length - 1);
}

private TreeNode build(int[] nums, int left, int right) {
    if (left > right) return null;
    int mid = left + (right - left) / 2;
    TreeNode root = new TreeNode(nums[mid]);
    root.left = build(nums, left, mid - 1);
    root.right = build(nums, mid + 1, right);
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(\log N)$ (balanced tree guaranteed)

109. Convert Sorted List to Binary Search Tree

Idea: Use slow/fast pointers to find the middle of the linked list.


```

public TreeNode sortedListToBST(ListNode head) {
    if (head == null) return null;
    if (head.next == null) return new TreeNode(head.val);

    ListNode slow = head, fast = head, prev = null;
    while (fast != null && fast.next != null) {
        prev = slow;
        slow = slow.next;
        fast = fast.next.next;
    }

    // Disconnect the left half
    prev.next = null;

    TreeNode root = new TreeNode(slow.val);
    root.left = sortedListToBST(head);
    root.right = sortedListToBST(slow.next);
    return root;
}

```

- **Complexity:** Time $O(N \log N)$ (can be $O(N)$ trickery with inorder iteration), Space $O(\log N)$

3. BST Iterators & Successors

173. Binary Search Tree Iterator

Idea: Use a stack to control the In-Order traversal manually.

```

class BSTIterator {
    private Stack<TreeNode> stack;
    public BSTIterator(TreeNode root) {
        stack = new Stack<>();
        pushAllLeft(root);
    }
    public int next() {
        TreeNode node = stack.pop();
        pushAllLeft(node.right);
        return node.val;
    }
    public boolean hasNext() {
        return !stack.isEmpty();
    }
    private void pushAllLeft(TreeNode node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }
}

```

- **Complexity:** Time $O(1)$ amortized per `next()`, Space $O(H)$ (Note for 1586. BST Iterator II: Combines this approach with an `ArrayList` to store visited nodes and a pointer, allowing `prev()` and `hasNext()` calls).

285. Inorder Successor in BST

Idea: If `root.val > p.val`, the successor could be `root` or in the left subtree. If `root.val <= p.val`, it must be in the right subtree.

```

public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
    TreeNode successor = null;
    while (root != null) {
        if (p.val >= root.val) {
            root = root.right;
        } else {
            successor = root;
            root = root.left;
        }
    }
    return successor;
}

```

- **Complexity:** Time $O(H)$, Space $O(1)$

510. Inorder Successor in BST II (Node has Parent Pointer)

*Idea: If node has a right child, successor is the leftmost node in right subtree. If no right child, traverse up the parent pointers until we are a **left** child of a parent. That parent is the successor.*

```

public Node inorderSuccessor(Node node) {
    if (node.right != null) {
        node = node.right;
        while (node.left != null) node = node.left;
        return node;
    }
    // Traverse up until the node is a left child of its parent
    while (node.parent != null && node == node.parent.right) {
        node = node.parent;
    }
    return node.parent;
}

```

- **Complexity:** Time $O(H)$, Space $O(1)$

4. More Practice & Advanced BST Invariants

96. Unique Binary Search Trees

Idea: DP / Catalan numbers. $G[n] = \sum (G[i-1] \times G[n-i])$ for $1 \leq i \leq n$.

```

public int numTrees(int n) {
    int[] dp = new int[n + 1];
    dp[0] = 1; dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            dp[i] += dp[j - 1] * dp[i - j];
        }
    }
    return dp[n];
}

```

- **Complexity:** Time $O(N^2)$, Space $O(N)$

95. Unique Binary Search Trees II

Idea: Recursively generate all left subtrees and all right subtrees, then connect them to all possible roots i .

```

public List<TreeNode> generateTrees(int n) {
    if (n == 0) return new ArrayList<>();
    return generate(1, n);
}
private List<TreeNode> generate(int start, int end) {
    List<TreeNode> res = new ArrayList<>();
    if (start > end) {
        res.add(null);
        return res;
    }
    for (int i = start; i <= end; i++) {
        List<TreeNode> leftTrees = generate(start, i - 1);
        List<TreeNode> rightTrees = generate(i + 1, end);
        for (TreeNode left : leftTrees) {
            for (TreeNode right : rightTrees) {
                TreeNode root = new TreeNode(i);
                root.left = left;
                root.right = right;
                res.add(root);
            }
        }
    }
    return res;
}

```

- **Complexity:** Time $O(4^N / N^{1.5})$ (Catalan number bounds), Space $O(4^N / N^{1.5})$

669. Trim a Binary Search Tree

Idea: If `root.val < low`, the root and left subtree are invalid, so return the trimmed right subtree. If `root.val > high`, return the trimmed left subtree.

```

public TreeNode trimBST(TreeNode root, int low, int high) {
    if (root == null) return null;
    if (root.val < low) return trimBST(root.right, low, high);
    if (root.val > high) return trimBST(root.left, low, high);

    root.left = trimBST(root.left, low, high);
    root.right = trimBST(root.right, low, high);
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

897. Increasing Order Search Tree

Idea: In-order traversal. Re-link the nodes into a right-skewed tree using a `dummy` node and `curr` pointer.

```

TreeNode curr;
public TreeNode increasingBST(TreeNode root) {
    TreeNode dummy = new TreeNode(0);
    curr = dummy;
    inorder(root);
    return dummy.right;
}
private void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    node.left = null; // Sever left pointer to prevent cycles!
    curr.right = node;
    curr = node;
    inorder(node.right);
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

449. Serialize and Deserialize BST

Idea: Exploit BST property. Preorder traversal for serialization (gives `[root, left, right]`). For deserialization, use an upper bound just like "Construct BST from Preorder". No need to store `null` markers like you do for a normal Binary Tree!

```
public class Codec {
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        preorder(root, sb);
        return sb.toString();
    }
    private void preorder(TreeNode root, StringBuilder sb) {
        if (root == null) return;
        sb.append(root.val).append(",");
        preorder(root.left, sb);
        preorder(root.right, sb);
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        if (data.isEmpty()) return null;
        String[] q = data.split(",");
        int[] pos = new int[1]; // Using array to simulate global pointer
        return build(q, pos, Integer.MAX_VALUE);
    }
    private TreeNode build(String[] q, int[] pos, int bound) {
        if (pos[0] == q.length || Integer.parseInt(q[pos[0]]) > bound) return null;
        TreeNode root = new TreeNode(Integer.parseInt(q[pos[0]++]));
        root.left = build(q, pos, root.val);
        root.right = build(q, pos, bound);
        return root;
    }
}
```

- **Complexity:** Time $O(N)$ for both, Space $O(N)$ for strings/arrays

Phase 3: Build / Recover / Serialize Trees

Why: Construction sharpens recursive thinking and index math. It tests your ability to map traversals back to subtrees.

1. Build from Traversals

105. Construct Binary Tree from Preorder and Inorder Traversal

Idea: `preorder[0]` is the root. Find it in `inorder` to split left and right subtrees. Use a `HashMap` for $O(1)$ lookups in `inorder` array.

```

Map<Integer, Integer> inMap = new HashMap<>();
int preIndex = 0;

public TreeNode buildTree(int[] preorder, int[] inorder) {
    for (int i = 0; i < inorder.length; i++) inMap.put(inorder[i], i);
    return build(preorder, 0, inorder.length - 1);
}

private TreeNode build(int[] preorder, int inStart, int inEnd) {
    if (inStart > inEnd) return null;
    TreeNode root = new TreeNode(preorder[preIndex++]);
    int inIndex = inMap.get(root.val); // Split point

    root.left = build(preorder, inStart, inIndex - 1);
    root.right = build(preorder, inIndex + 1, inEnd);
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

106. Construct Binary Tree from Inorder and Postorder Traversal

*Idea: `postorder[last]` is the root. Traverse `postorder` **backwards** (`postIndex--`). Build `right` subtree **FIRST**, then `left` subtree, because `postorder` is [`left`, `right`, `ROOT`].*

```

Map<Integer, Integer> inMap = new HashMap<>();
int postIndex;

public TreeNode buildTree(int[] inorder, int[] postorder) {
    postIndex = postorder.length - 1;
    for (int i = 0; i < inorder.length; i++) inMap.put(inorder[i], i);
    return build(postorder, 0, inorder.length - 1);
}

private TreeNode build(int[] postorder, int inStart, int inEnd) {
    if (inStart > inEnd) return null;
    TreeNode root = new TreeNode(postorder[postIndex--]);
    int inIndex = inMap.get(root.val);

    root.right = build(postorder, inIndex + 1, inEnd); // RIGHT FIRST!
    root.left = build(postorder, inStart, inIndex - 1);
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

889. Construct Binary Tree from Preorder and Postorder Traversal

Idea: `preorder[0]` is root. `preorder[1]` is left child root. Traverse implicitly without tracking exact splits by maintaining pointers.

```

int preIndex = 0, postIndex = 0;
public TreeNode constructFromPrePost(int[] preorder, int[] postorder) {
    TreeNode root = new TreeNode(preorder[preIndex++]);
    if (root.val != postorder[postIndex]) { // Left child exists
        root.left = constructFromPrePost(preorder, postorder);
    }
    if (root.val != postorder[postIndex]) { // Right child exists
        root.right = constructFromPrePost(preorder, postorder);
    }
    postIndex++;
    return root;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

2. Recover & Flatten

1028. Recover a Tree From Preorder Traversal

Idea: The number of dashes - represents the depth. Compare the current depth with expected depth using a single global pointer `i`.

```
int i = 0;
public TreeNode recoverFromPreorder(String S) {
    return dfs(S, 0);
}
private TreeNode dfs(String S, int depth) {
    int numDashes = 0;
    while (i + numDashes < S.length() && S.charAt(i + numDashes) == '-') {
        numDashes++;
    }
    if (numDashes != depth) return null;

    i += numDashes;
    int val = 0;
    while (i < S.length() && Character.isDigit(S.charAt(i))) {
        val = val * 10 + (S.charAt(i) - '0');
        i++;
    }

    TreeNode root = new TreeNode(val);
    root.left = dfs(S, depth + 1);
    root.right = dfs(S, depth + 1);
    return root;
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

114. Flatten Binary Tree to Linked List

*Idea: Preorder threading into a right-skewed tree. Track the `prev` node using a **Reverse Post-Order** traversal (`Right -> Left -> Root`).*

```
TreeNode prev = null;
public void flatten(TreeNode root) {
    if (root == null) return;
    flatten(root.right);
    flatten(root.left);

    root.right = prev;
    root.left = null;
    prev = root;
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

3. Serialization & Deserialization

297. Serialize and Deserialize Binary Tree (DFS)

Idea: Preorder traversal using `,` as delimiter and `X` for nulls.

```

public class Codec {
    public String serialize(TreeNode root) {
        if (root == null) return "X,";
        return root.val + "," + serialize(root.left) + serialize(root.right);
    }

    public TreeNode deserialize(String data) {
        Queue<String> q = new LinkedList<>(Arrays.asList(data.split(",")));
        return build(q);
    }

    private TreeNode build(Queue<String> q) {
        String val = q.poll();
        if (val.equals("X")) return null;

        TreeNode root = new TreeNode(Integer.parseInt(val));
        root.left = build(q);
        root.right = build(q);
        return root;
    }
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$ (Note: As documented in Phase 2, 449. *Serialize BST* is identical but removes the *X* null markers by feeding an upper integer bound during deserialization instead, exploiting the sorting property).

606. Construct String from Binary Tree

*Idea: Preorder traversal. Wrap subtrees in `()`. Omit empty pairs `()` unless the left child is null but the right child exists (then we **MUST** keep `()` for the left side to maintain correct positional structure).*

```

public String tree2str(TreeNode root) {
    if (root == null) return "";
    if (root.left == null && root.right == null) return String.valueOf(root.val);
    if (root.right == null) return root.val + "(" + tree2str(root.left) + ")";

    return root.val + "(" + tree2str(root.left) + ")(" + tree2str(root.right) + ")";
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$ (string allocations can make it behave worse without `StringBuilder`)

536. Construct Binary Tree from String

Idea: String format like `4(2(3)(1))(6(5))`. Find matched parentheses and recursively call build on left and right substrings.

```

public TreeNode str2tree(String s) {
    if (s == null || s.length() == 0) return null;
    int firstParen = s.indexOf("(");
    if (firstParen == -1) return new TreeNode(Integer.parseInt(s)); // No children

    TreeNode root = new TreeNode(Integer.parseInt(s.substring(0, firstParen)));
    int start = firstParen, count = 0;

    // Find matching parenthesis for left child
    for (int i = start; i < s.length(); i++) {
        if (s.charAt(i) == '(') count++;
        else if (s.charAt(i) == ')') count--;

        if (count == 0 && start == firstParen) {
            root.left = str2tree(s.substring(start + 1, i)); // Left child
            start = i + 1; // Start of right child (if any)
        } else if (count == 0) {
            root.right = str2tree(s.substring(start + 1, i)); // Right child
        }
    }
    return root;
}

```

- **Complexity:** Time $O(N^2)$ worst-case substring/indexOf, Space $O(N)$ call stack / sub-strings. (Can be optimized to $O(N)$ Time / $O(H)$ space using a global pointer like 1028. Recover Tree above)

4. More Practice & Complete Binary Trees

222. Count Complete Tree Nodes

Idea: A complete binary tree has $2^h - 1$ nodes if left and right subtrees have the same height. Otherwise, we recursively sum $1 + \text{count}(\text{left}) + \text{count}(\text{right})$.

```

public int countNodes(TreeNode root) {
    if (root == null) return 0;
    int leftHeight = getLeftHeight(root);
    int rightHeight = getRightHeight(root);

    // If heights match, it is a perfect binary tree
    if (leftHeight == rightHeight) {
        return (1 << leftHeight) - 1; // 2^h - 1
    }
    // Otherwise, normal recursive count
    return 1 + countNodes(root.left) + countNodes(root.right);
}

private int getLeftHeight(TreeNode root) {
    int h = 0;
    while (root != null) { h++; root = root.left; }
    return h;
}

private int getRightHeight(TreeNode root) {
    int h = 0;
    while (root != null) { h++; root = root.right; }
    return h;
}

```

- **Complexity:** Time $O(\log^2 N)$ (Height computation takes $O(\log N)$, done at most $O(\log N)$ times), Space $O(\log N)$

958. Check Completeness of a Binary Tree

Idea: Level-order traversal (BFS). Once we see a `null` node, ALL subsequent nodes in the queue MUST be `null` for the tree to be complete.

```
public boolean isCompleteTree(TreeNode root) {
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    boolean seenNull = false;

    while (!q.isEmpty()) {
        TreeNode curr = q.poll();
        if (curr == null) {
            seenNull = true;
        } else {
            if (seenNull) return false; // Saw a node AFTER seeing a null!
            q.offer(curr.left);
            q.offer(curr.right);
        }
    }
    return true;
}
```

- **Complexity:** Time $O(N)$, Space $O(N)$

919. Complete Binary Tree Inserter

Idea: Maintain a `Queue` of nodes that are missing at least one child (i.e., potential parents). To insert: attach to `peek()`, then add the new node to the queue. If `peek()` now has both children, `poll()` it.

```

class CBTInserter {
    private TreeNode root;
    private Queue<TreeNode> q; // Stores nodes that don't have 2 children yet

    public CBTInserter(TreeNode root) {
        this.root = root;
        this.q = new LinkedList<>();

        // Standard BFS to populate 'q' with incomplete nodes
        Queue<TreeNode> initQ = new LinkedList<>();
        initQ.offer(root);
        while (!initQ.isEmpty()) {
            TreeNode curr = initQ.poll();
            if (curr.left == null || curr.right == null) {
                q.offer(curr); // This node can accept children
            }
            if (curr.left != null) initQ.offer(curr.left);
            if (curr.right != null) initQ.offer(curr.right);
        }
    }

    public int insert(int val) {
        TreeNode parent = q.peek();
        TreeNode newNode = new TreeNode(val);
        q.offer(newNode); // New node will eventually need children too

        if (parent.left == null) {
            parent.left = newNode;
        } else {
            parent.right = newNode;
            q.poll(); // Parent is now full! Remove it.
        }
        return parent.val;
    }

    public TreeNode get_root() {
        return root;
    }
}

```

- **Complexity:** Time $O(N)$ for init, $O(1)$ for insert/get, Space $O(N)$

Phase 4: Paths, Sums, & DP on Trees

Why: These medium-to-hard problems test your ability to use postorder traversal to pass states up, or preorder traversal to pass states down (like prefix sums and bitmasks).

1. Path Sums & Root-to-Leaf

112. Path Sum I (Exists?)

Idea: Subtract node value from target. Base case: leaf node matches remaining sum.

```
public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false;
    if (root.left == null && root.right == null) return targetSum == root.val;

    int rem = targetSum - root.val;
    return hasPathSum(root.left, rem) || hasPathSum(root.right, rem);
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

113. Path Sum II (All Paths)

Idea: Backtracking DFS. Add node to `path`, recurse, then remove node from `path`.

```
public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
    List<List<Integer>> res = new ArrayList<>();
    dfs(root, targetSum, new ArrayList<>(), res);
    return res;
}

private void dfs(TreeNode root, int sum, List<Integer> path, List<List<Integer>> res) {
    if (root == null) return;
    path.add(root.val);

    if (root.left == null && root.right == null && sum == root.val) {
        res.add(new ArrayList<>(path));
    } else {
        dfs(root.left, sum - root.val, path, res);
        dfs(root.right, sum - root.val, path, res);
    }

    path.remove(path.size() - 1); // Backtrack
}
```

- **Complexity:** Time $O(N^2)$ (due to copying paths to result), Space $O(H)$

437. Path Sum III (Prefix Sum HashMap)

Idea: Count paths that sum to target starting from any node going downwards. Use a HashMap of `prefix_sum -> count` (like Subarray Sum Equals K).

```
public int pathSum(TreeNode root, int targetSum) {
    Map<Long, Integer> map = new HashMap<>();
    map.put(0L, 1); // Base case: prefix sum equals target
    return dfs(root, 0L, targetSum, map);
}

private int dfs(TreeNode root, long currSum, int target, Map<Long, Integer> map) {
    if (root == null) return 0;

    currSum += root.val;
    int count = map.getOrDefault(currSum - target, 0);

    map.put(currSum, map.getOrDefault(currSum, 0) + 1);

    count += dfs(root.left, currSum, target, map);
    count += dfs(root.right, currSum, target, map);

    // Backtrack the prefix sum for parallel branches
    map.put(currSum, map.get(currSum) - 1);

    return count;
}
```

- **Complexity:** Time $O(N)$, Space $O(N)$

129. Sum Root to Leaf Numbers

Idea: Pass the running number down as `curr = curr * 10 + root.val` .

```
public int sumNumbers(TreeNode root) {
    return dfs(root, 0);
}
private int dfs(TreeNode root, int curr) {
    if (root == null) return 0;
    curr = curr * 10 + root.val;
    if (root.left == null && root.right == null) return curr;
    return dfs(root.left, curr) + dfs(root.right, curr);
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

1022. Sum of Root To Leaf Binary Numbers

Idea: Same as 129, but base 2: `curr = (curr << 1) | root.val` .

```
public int sumRootToLeaf(TreeNode root) {
    return dfs(root, 0);
}
private int dfs(TreeNode root, int curr) {
    if (root == null) return 0;
    curr = (curr << 1) | root.val;
    if (root.left == null && root.right == null) return curr;
    return dfs(root.left, curr) + dfs(root.right, curr);
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

2. Maximum Gains & DP on Trees

124. Binary Tree Maximum Path Sum

Idea: Postorder traversal. Return the maximum "straight path" gain to the parent (`max(left, right) + root.val`). Update the `globalBest` path passing through the current node (`left + right + root.val`). *Math.max with 0 to ignore negative subtrees!*

```
int maxPath = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) {
    dfsGain(root);
    return maxPath;
}
private int dfsGain(TreeNode root) {
    if (root == null) return 0;

    // Max of 0 ignores negative paths
    int leftGain = Math.max(0, dfsGain(root.left));
    int rightGain = Math.max(0, dfsGain(root.right));

    // Price of path passing THROUGH root (bridging the tree)
    maxPath = Math.max(maxPath, leftGain + rightGain + root.val);

    // Return max gain extending from root down ONE side
    return Math.max(leftGain, rightGain) + root.val;
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

543. Diameter of Binary Tree

Idea: Identical pattern to Max Path Sum. Return height to parent. Update `globalMax` with `leftHeight + rightHeight`.

```
int maxDiameter = 0;
public int diameterOfBinaryTree(TreeNode root) {
    dfsHeight(root);
    return maxDiameter;
}
private int dfsHeight(TreeNode root) {
    if (root == null) return 0;

    int left = dfsHeight(root.left);
    int right = dfsHeight(root.right);

    maxDiameter = Math.max(maxDiameter, left + right); // Edges between leaves

    return Math.max(left, right) + 1; // Height of current node
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

687. Longest Univalue Path

Idea: Postorder. Calculate the longest univalue path extending from left/right children. If child value matches root, extend the path by 1. Otherwise, it breaks (becomes 0).

```
int maxLen = 0;
public int longestUnivaluePath(TreeNode root) {
    dfs(root);
    return maxLen;
}
private int dfs(TreeNode root) {
    if (root == null) return 0;

    int left = dfs(root.left);
    int right = dfs(root.right);

    int leftArrow = 0, rightArrow = 0;
    if (root.left != null && root.left.val == root.val) leftArrow = left + 1;
    if (root.right != null && root.right.val == root.val) rightArrow = right + 1;

    maxLen = Math.max(maxLen, leftArrow + rightArrow); // Bridge the univalue paths
    return Math.max(leftArrow, rightArrow);          // Return longest valid branch
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

3. Subtrees & Bitmask State

865 / 1123. Smallest Subtree with all Deepest Nodes (LCA of Deepest)

Idea: Postorder. Return a Custom `Result` pair (`node`, `depth`). If `left.depth == right.depth`, current node is LCA. Otherwise, pass up the side with deeper nodes.

```

class Result {
    TreeNode node;
    int depth;
    Result(TreeNode n, int d) { node = n; depth = d; }
}

public TreeNode lcaDeepestLeaves(TreeNode root) {
    return dfs(root, 0).node;
}

private Result dfs(TreeNode root, int depth) {
    if (root == null) return new Result(null, depth);

    Result left = dfs(root.left, depth + 1);
    Result right = dfs(root.right, depth + 1);

    if (left.depth == right.depth) {
        return new Result(root, left.depth); // Found LCA for this depth locally
    }
    return left.depth > right.depth ? left : right; // Pass up the deeper side
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

1457. Pseudo-Palindromic Paths in a Binary Tree

*Idea: A path is pseudo-palindromic if at most **one** digit has an odd frequency. Track the parity of digits using a 10-bit integer bitmask. XOR toggles frequency parity.*

```

public int pseudoPalindromicPaths (TreeNode root) {
    return dfs(root, 0);
}

private int dfs(TreeNode root, int mask) {
    if (root == null) return 0;

    // Toggle the bit for this digit: 1 << root.val
    mask ^= (1 << root.val);

    if (root.left == null && root.right == null) {
        // At leaf: Check if at most one bit is set.
        // Trick: (mask & (mask - 1)) clears the lowest set bit. If 0, then <= 1 bits were set.
        return (mask & (mask - 1)) == 0 ? 1 : 0;
    }

    return dfs(root.left, mask) + dfs(root.right, mask);
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

4. More Practice: Tree DP & Sequences

979. Distribute Coins in Binary Tree

Idea: Postorder DP. The number of moves required to balance a subtree is $\text{abs}(\text{coins} - \text{nodes})$. Return the "balance" to the parent ($\text{left} + \text{right} + \text{root.val} - 1$). Total moves is the sum of the absolute balances flowing through every edge.

```

int moves = 0;
public int distributeCoins(TreeNode root) {
    dfs(root);
    return moves;
}

private int dfs(TreeNode root) {
    if (root == null) return 0;

    int leftFlow = dfs(root.left);
    int rightFlow = dfs(root.right);

    moves += Math.abs(leftFlow) + Math.abs(rightFlow);

    // Balance: positive means excess coins to push up, negative means need coins
    return root.val + leftFlow + rightFlow - 1;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

1339. Maximum Product of Splitted Binary Tree

*Idea: Two passes. First pass: find the total sum of the tree. Second pass (or using saved subtree sums): compute $subtreeSum * (totalSum - subtreeSum)$ for every subtree and track the max.*

```

long maxProd = 0;
long totalSum = 0;

public int maxProduct(TreeNode root) {
    totalSum = getSum(root); // Pass 1
    getSum(root);           // Pass 2 to calculate products
    return (int) (maxProd % 1_000_000_007);
}

private long getSum(TreeNode root) {
    if (root == null) return 0;
    long currSum = root.val + getSum(root.left) + getSum(root.right);

    long product = currSum * (totalSum - currSum);
    maxProd = Math.max(maxProd, product);

    return currSum;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

508. Most Frequent Subtree Sum

Idea: Postorder to calculate all subtree sums. Use a HashMap to count frequencies and track $maxFreq$.

```

Map<Integer, Integer> countMap = new HashMap<>();
int maxCount = 0;

public int[] findFrequentTreeSum(TreeNode root) {
    dfs(root);
    List<Integer> res = new ArrayList<>();
    for (int key : countMap.keySet()) {
        if (countMap.get(key) == maxCount) res.add(key);
    }
    return res.stream().mapToInt(i -> i).toArray();
}

private int dfs(TreeNode root) {
    if (root == null) return 0;
    int sum = root.val + dfs(root.left) + dfs(root.right);

    int count = countMap.getOrDefault(sum, 0) + 1;
    countMap.put(sum, count);
    maxCount = Math.max(maxCount, count);

    return sum;
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

666. Path Sum IV

*Idea: Tree is given as an array of 3-digit ints (depth, position, value). Use a HashMap where $key = 10 * depth + pos$. A node's left child is $10 * (depth + 1) + (2 * pos - 1)$, right is $10 * (depth + 1) + (2 * pos)$.*

```

int sum = 0;
Map<Integer, Integer> map = new HashMap<>();

public int pathSum(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    for (int num : nums) {
        map.put(num / 10, num % 10);
    }
    dfs(nums[0] / 10, 0);
    return sum;
}

private void dfs(int nodePos, int currSum) {
    if (!map.containsKey(nodePos)) return;

    int depth = nodePos / 10, pos = nodePos % 10;
    currSum += map.get(nodePos);

    int leftPos = 10 * (depth + 1) + 2 * pos - 1;
    int rightPos = 10 * (depth + 1) + 2 * pos;

    // Found a leaf: no left AND no right children matching calculated keys
    if (!map.containsKey(leftPos) && !map.containsKey(rightPos)) {
        sum += currSum;
        return;
    }

    dfs(leftPos, currSum);
    dfs(rightPos, currSum);
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

298. Binary Tree Longest Consecutive Sequence

Idea: Preorder DFS. Pass down the `expected_value` and the `current_length`. Update a `globalMax`.

```
int maxLength = 0;
public int longestConsecutive(TreeNode root) {
    if (root == null) return 0;
    dfs(root, root.val, 0); // Need to seed initial expectation safely
    return maxLength;
}

private void dfs(TreeNode root, int expected, int curLen) {
    if (root == null) return;

    if (root.val == expected) curLen++;
    else curLen = 1; // Sequence broken, reset length

    maxLength = Math.max(maxLength, curLen);

    dfs(root.left, root.val + 1, curLen);
    dfs(root.right, root.val + 1, curLen);
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

549. Binary Tree Longest Consecutive Sequence II

Idea: Postorder. Each node returns `[inc, dec]`, the longest increasing and decreasing contiguous sequences starting at this root moving down. The longest path spanning ACROSS root is `inc + dec - 1`.

```
int maxLen = 0;
public int longestConsecutive(TreeNode root) {
    dfs(root);
    return maxLen;
}

// Returns [inc, dec]
private int[] dfs(TreeNode root) {
    if (root == null) return new int[]{0, 0};

    int inc = 1, dec = 1;
    int[] left = dfs(root.left);
    int[] right = dfs(root.right);

    if (root.left != null) {
        if (root.val == root.left.val + 1) dec = Math.max(dec, left[1] + 1);
        if (root.val == root.left.val - 1) inc = Math.max(inc, left[0] + 1);
    }
    if (root.right != null) {
        if (root.val == root.right.val + 1) dec = Math.max(dec, right[1] + 1);
        if (root.val == root.right.val - 1) inc = Math.max(inc, right[0] + 1);
    }

    maxLen = Math.max(maxLen, inc + dec - 1);
    return new int[]{inc, dec};
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

Phase 5: Graphy Trees, LCA Variants, & Views

Why: These problems bridge the gap between Trees and pure Graphs. You'll learn how to treat trees as undirected graphs, serialize structures for deep comparison, and master 2D spatial coordinate mapping.

1. LCA Masterclass

236. Lowest Common Ancestor of a Binary Tree (General)

Idea: Postorder. If the current root is `p` or `q`, return it. If both left and right return non-null, the current root is the LCA. If only one returns non-null, pass it up.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root; // Found one!

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) return root;           // Split point! This is LCA
    return left != null ? left : right;                       // Pass up the found node
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

1644. Lowest Common Ancestor II (Nodes Might Not Exist)

Idea: You CANNOT return early if you find `p` or `q` because the other node might not exist in the subtree. You MUST always search the entire tree context by placing the return after exploration.

```
boolean foundP = false, foundQ = false;
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    TreeNode lca = dfs(root, p, q);
    return (foundP && foundQ) ? lca : null;
}
private TreeNode dfs(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) return null;

    TreeNode left = dfs(root.left, p, q);
    TreeNode right = dfs(root.right, p, q);

    if (root == p) { foundP = true; return root; }
    if (root == q) { foundQ = true; return root; }

    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

- **Complexity:** Time $O(N)$, Space $O(H)$

1650. Lowest Common Ancestor III (Given Parent Pointers)

Idea: It's exactly the intersection of two linked lists! Walk up using `parent` pointers. If they don't match, swap to the other node's start when reaching `null`.

```
public Node lowestCommonAncestor(Node p, Node q) {
    Node a = p, b = q;
    while (a != b) {
        a = a == null ? q : a.parent;
        b = b == null ? p : b.parent;
    }
    return a;
}
```

- **Complexity:** Time $O(H)$, Space $O(1)$

1676. Lowest Common Ancestor IV (Array of Nodes)

Idea: Put target nodes in a `HashSet`. Standard LCA logic applies: if `set.contains(root)`, return it.

```

public TreeNode lowestCommonAncestor(TreeNode root, TreeNode[] nodes) {
    Set<TreeNode> set = new HashSet<>(Arrays.asList(nodes));
    return dfs(root, set);
}

private TreeNode dfs(TreeNode root, Set<TreeNode> set) {
    if (root == null || set.contains(root)) return root;

    TreeNode left = dfs(root.left, set);
    TreeNode right = dfs(root.right, set);

    if (left != null && right != null) return root;
    return left != null ? left : right;
}

```

- **Complexity:** Time $O(N)$, Space $O(H + K)$ where K is size of nodes array

2. Graph Conversions & Pathways

2096. Step-By-Step Directions From Start to Destination

Idea: Find the LCA. Find the path from LCA to `startValue` (all U s). Find the path from LCA to `destValue` (using L and R). Combine them.

```

public String getDirections(TreeNode root, int startValue, int destValue) {
    StringBuilder sPath = new StringBuilder();
    StringBuilder dPath = new StringBuilder();

    findPath(root, startValue, sPath);
    findPath(root, destValue, dPath);

    // Remove common prefix (the path from root down to their LCA)
    int i = 0;
    while (i < sPath.length() && i < dPath.length() && sPath.charAt(i) == dPath.charAt(i)) i++;

    // Replace the remainder of the start path with 'U's (moving UP to LCA)
    StringBuilder res = new StringBuilder();
    for (int j = i; j < sPath.length(); j++) res.append('U');

    // Append the remainder of the destination path (moving DOWN from LCA)
    res.append(dPath.substring(i));
    return res.toString();
}

private boolean findPath(TreeNode root, int target, StringBuilder path) {
    if (root == null) return false;
    if (root.val == target) return true;

    path.append('L');
    if (findPath(root.left, target, path)) return true;
    path.deleteCharAt(path.length() - 1); // Backtrack

    path.append('R');
    if (findPath(root.right, target, path)) return true;
    path.deleteCharAt(path.length() - 1); // Backtrack

    return false;
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$ for stringbuilders / call stack

863. All Nodes Distance K in Binary Tree

Idea: Since we need to go "up" to ancestors, attach `parent` pointers via DFS mapping, effectively turning the tree into an undirected graph. Then do a standard BFS starting from `target` node up to distance `K`.

```
Map<TreeNode, TreeNode> parentMap = new HashMap<>();

public List<Integer> distanceK(TreeNode root, TreeNode target, int k) {
    buildGraph(root, null);

    Queue<TreeNode> q = new LinkedList<>();
    Set<TreeNode> visited = new HashSet<>();
    q.offer(target);
    visited.add(target);

    int dist = 0;
    List<Integer> res = new ArrayList<>();

    while (!q.isEmpty()) {
        if (dist == k) {
            for (TreeNode n : q) res.add(n.val);
            return res;
        }
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();

            // Standard Graph Traversal (Left, Right, and Parent!)
            if (curr.left != null && visited.add(curr.left)) q.offer(curr.left);
            if (curr.right != null && visited.add(curr.right)) q.offer(curr.right);

            TreeNode parent = parentMap.get(curr);
            if (parent != null && visited.add(parent)) q.offer(parent);
        }
        dist++;
    }
    return res;
}

private void buildGraph(TreeNode node, TreeNode parent) {
    if (node == null) return;
    parentMap.put(node, parent);
    buildGraph(node.left, node);
    buildGraph(node.right, node);
}
```

- **Complexity:** Time $O(N)$, Space $O(N)$ (graph map + queue)

257. Binary Tree Paths

Idea: Classic Preorder DFS top-down. Avoid String building overhead in recursion unless passing down current state.

```
public List<String> binaryTreePaths(TreeNode root) {
    List<String> res = new ArrayList<>();
    if (root != null) dfs(root, "", res);
    return res;
}

private void dfs(TreeNode root, String path, List<String> res) {
    if (root.left == null && root.right == null) res.add(path + root.val);
    if (root.left != null) dfs(root.left, path + root.val + "->", res);
    if (root.right != null) dfs(root.right, path + root.val + "->", res);
}
```

- **Complexity:** Time $O(N)$ technically string allocs make it $O(N \log N)$ average, Space $O(N)$

3. Structural Comparison & Coordinate Mapping

572. Subtree of Another Tree

Idea: At every node in `root`, check if `isSameTree(node, subRoot)`.

```
public boolean isSubtree(TreeNode root, TreeNode subRoot) {
    if (root == null) return false;
    if (isSameTree(root, subRoot)) return true;
    return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
}

private boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

- **Complexity:** Time $O(N \times M)$, Space $O(\max(H_n, H_m))$

652. Find Duplicate Subtrees

Idea: Serialize each subtree to a string (Preorder/Postorder). Count the frequency of each serialization in a HashMap. If `count == 2`, it's exactly the first time we realized there's a duplicate pair! Add the node.

```
Map<String, Integer> count = new HashMap<>();
List<TreeNode> res = new ArrayList<>();

public List<TreeNode> findDuplicateSubtrees(TreeNode root) {
    serialize(root);
    return res;
}

private String serialize(TreeNode node) {
    if (node == null) return "#";
    // Subtree identity hash
    String serial = node.val + "," + serialize(node.left) + "," + serialize(node.right);

    count.put(serial, count.getOrDefault(serial, 0) + 1);

    // Add exactly once per duplicate pair
    if (count.get(serial) == 2) {
        res.add(node);
    }
    return serial;
}
```

- **Complexity:** Time $O(N^2)$ (due to string concats), Space $O(N^2)$ (Can be $O(N)$ with custom ID encoding)

987 / 314. Vertical Order Traversal of a Binary Tree

Idea: Assign coordinates `(row, col)` to each node. Left child is `(r+1, c-1)`, right is `(r+1, c+1)`. For #987, order requires sorting by `col` -> `row` -> `value`. Use deeply nested maps + priority queue.

```

class Point {
    TreeNode node; int row; int col;
    Point(TreeNode n, int r, int c) { node = n; row = r; col = c; }
}

public List<List<Integer>> verticalTraversal(TreeNode root) {
    // Map<col, Map<row, PriorityQueue<Values>>>
    TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>> map = new TreeMap<>();
    Queue<Point> q = new LinkedList<>();
    q.offer(new Point(root, 0, 0));

    while (!q.isEmpty()) {
        Point p = q.poll();
        map.putIfAbsent(p.col, new TreeMap<>());
        map.get(p.col).putIfAbsent(p.row, new PriorityQueue<>());
        map.get(p.col).get(p.row).offer(p.node.val);

        if (p.node.left != null) q.offer(new Point(p.node.left, p.row + 1, p.col - 1));
        if (p.node.right != null) q.offer(new Point(p.node.right, p.row + 1, p.col + 1));
    }

    List<List<Integer>> res = new ArrayList<>();
    for (TreeMap<Integer, PriorityQueue<Integer>> ys : map.values()) {
        List<Integer> colNodes = new ArrayList<>();
        // Extract row priorities iteratively down that column
        for (PriorityQueue<Integer> nodes : ys.values()) {
            while (!nodes.isEmpty()) colNodes.add(nodes.poll());
        }
        res.add(colNodes);
    }
    return res;
}

```

(Note: #314 Binary Tree Vertical Order Traversal doesn't strictly sort by row or value internally. It just requires a `HashMap<col, List>` tracking `minCol` and `maxCol`, heavily dropping complexity).

- **Complexity:** Time $O(N \log N)$ (TreeMap and PQ sorts), Space $O(N)$

4. More Practice: Boundary, Leaves, & Searches

742. Closest Leaf in a Binary Tree

Idea: Build an undirected graph using parent pointers (exactly like 863. Distance K). Then run a BFS starting from `k` until you hit ANY leaf node (a node with no left and no right child in the original tree).

```

public int findClosestLeaf(TreeNode root, int k) {
    Map<TreeNode, TreeNode> parentMap = new HashMap<>();
    TreeNode startNode = buildGraphAndFindStart(root, null, k, parentMap);

    Queue<TreeNode> q = new LinkedList<>();
    Set<TreeNode> visited = new HashSet<>();
    q.offer(startNode);
    visited.add(startNode);

    while (!q.isEmpty()) {
        TreeNode curr = q.poll();
        if (curr.left == null && curr.right == null) return curr.val; // First leaf found!

        if (curr.left != null && visited.add(curr.left)) q.offer(curr.left);
        if (curr.right != null && visited.add(curr.right)) q.offer(curr.right);

        TreeNode parent = parentMap.get(curr);
        if (parent != null && visited.add(parent)) q.offer(parent);
    }
    return -1;
}

private TreeNode buildGraphAndFindStart(TreeNode node, TreeNode parent, int k, Map<TreeNode, TreeNode> map) {
    if (node == null) return null;
    map.put(node, parent);
    if (node.val == k) {
        // Build rest of the graph
        buildGraphAndFindStart(node.left, node, k, map);
        buildGraphAndFindStart(node.right, node, k, map);
        return node;
    }
    TreeNode left = buildGraphAndFindStart(node.left, node, k, map);
    if (left != null) return left;
    return buildGraphAndFindStart(node.right, node, k, map);
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

545. Boundary of Binary Tree

Idea: Break it into 3 parts: Left Boundary (Top-Down Preorder, excluding leaves), Leaves (standard DFS), Right Boundary (Bottom-Up Postorder, excluding leaves). Root is handled separately.

```

public List<Integer> boundaryOfBinaryTree(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    res.add(root.val);
    if (root.left == null && root.right == null) return res;

    addLeft(root.left, res);
    addLeaves(root, res);
    addRight(root.right, res);
    return res;
}

private void addLeft(TreeNode node, List<Integer> res) {
    if (node == null || isLeaf(node)) return;
    res.add(node.val); // Pre-order
    if (node.left != null) addLeft(node.left, res);
    else addLeft(node.right, res);
}

private void addRight(TreeNode node, List<Integer> res) {
    if (node == null || isLeaf(node)) return;
    if (node.right != null) addRight(node.right, res);
    else addRight(node.left, res);
    res.add(node.val); // Post-order (reverses it natively!)
}

private void addLeaves(TreeNode node, List<Integer> res) {
    if (node == null) return;
    if (isLeaf(node)) res.add(node.val);
    addLeaves(node.left, res);
    addLeaves(node.right, res);
}

private boolean isLeaf(TreeNode node) {
    return node != null && node.left == null && node.right == null;
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

671. Second Minimum Node In a Binary Tree

Idea: Problem states $root.val = \min(root.left.val, root.right.val)$. Therefore, root is always the absolute minimum. We simply DFS to find the smallest value that is strictly greater than $root.val$.

```

int min1;
long min2 = Long.MAX_VALUE;

public int findSecondMinimumValue(TreeNode root) {
    min1 = root.val;
    dfs(root);
    return min2 < Long.MAX_VALUE ? (int) min2 : -1;
}

private void dfs(TreeNode root) {
    if (root == null) return;
    if (root.val > min1 && root.val < min2) {
        min2 = root.val;
    } else if (root.val == min1) {
        dfs(root.left);
        dfs(root.right);
    }
}

```

- **Complexity:** Time $O(N)$, Space $O(H)$

872. Leaf-Similar Trees

Idea: DFS to collect leaf values into lists. Compare the lists.

```
public boolean leafSimilar(TreeNode root1, TreeNode root2) {
    List<Integer> list1 = new ArrayList<>();
    List<Integer> list2 = new ArrayList<>();
    dfs(root1, list1);
    dfs(root2, list2);
    return list1.equals(list2);
}

private void dfs(TreeNode root, List<Integer> list) {
    if (root == null) return;
    if (root.left == null && root.right == null) list.add(root.val);
    dfs(root.left, list);
    dfs(root.right, list);
}
```

- **Complexity:** Time $O(N_1 + N_2)$, Space $O(L_1 + L_2)$ where L is number of leaves.

270. Closest Binary Search Tree Value

Idea: Traverse the BST (binary search). Maintain a `closest` variable, updating it as you plunge left or right depending on `target`'s relation to `root.val`.

```
public int closestValue(TreeNode root, double target) {
    int closest = root.val;
    while (root != null) {
        if (Math.abs(root.val - target) < Math.abs(closest - target) ||
            (Math.abs(root.val - target) == Math.abs(closest - target) && root.val < closest)) {
            closest = root.val;
        }
        root = target < root.val ? root.left : root.right;
    }
    return closest;
}
```

- **Complexity:** Time $O(H)$, Space $O(1)$

272. Closest Binary Search Tree Value II

Idea: In-order traversal to populate a `LinkedList` (Queue). If adding the new node keeps the queue size $\leq k$, compare new node with `queue.peekFirst()`. If it's closer, `pollFirst()` and `addLast()`. Otherwise, we can stop the overall traversal entirely because distances will only increase!

```

public List<Integer> closestKValues(TreeNode root, double target, int k) {
    LinkedList<Integer> queue = new LinkedList<>();
    inorder(root, target, k, queue);
    return queue;
}

private void inorder(TreeNode root, double target, int k, LinkedList<Integer> q) {
    if (root == null) return;

    inorder(root.left, target, k, q);

    if (q.size() < k) {
        q.addLast(root.val);
    } else {
        if (Math.abs(root.val - target) < Math.abs(q.peekFirst() - target)) {
            q.removeFirst();
            q.addLast(root.val);
        } else {
            return; // We can stop! Future nodes will only be further away!
        }
    }

    inorder(root.right, target, k, q);
}

```

- **Complexity:** Time $O(N)$ (often much less due to early exit), Space $O(H + K)$

2476. Closest Nodes Queries in a Binary Search Tree

Idea: The BST might NOT be balanced ($O(N)$ lookup per query would TLE). In-order traverse once to get a sorted array ($O(N)$). For each query, binarySearch the array ($O(\log N)$) to find the strict floor and ceiling.

```

public List<List<Integer>> closestNodes(TreeNode root, List<Integer> queries) {
    List<Integer> sorted = new ArrayList<>();
    inorder(root, sorted); // O(N)

    List<List<Integer>> res = new ArrayList<>();
    for (int q : queries) {
        res.add(binarySearch(sorted, q)); // O(log N) per query
    }
    return res;
}

private void inorder(TreeNode node, List<Integer> sorted) {
    if (node == null) return;
    inorder(node.left, sorted);
    sorted.add(node.val);
    inorder(node.right, sorted);
}

private List<Integer> binarySearch(List<Integer> a, int target) {
    int left = 0, right = a.size() - 1;
    int minObj = -1, maxObj = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (a.get(mid) == target) {
            return Arrays.asList(target, target);
        } else if (a.get(mid) < target) {
            minObj = a.get(mid);
            left = mid + 1;
        } else {
            maxObj = a.get(mid);
            right = mid - 1;
        }
    }
    return Arrays.asList(minObj, maxObj);
}

```

- **Complexity:** Time $O(N + Q \log N)$, Space $O(N)$

Phase 6: Advanced Tree DP & Rerooting

Why: This is the SDE-2 / SDE-3 true separator. It involves converting unrooted trees into rooted ones, processing DP in two passes (down then up), keeping heavy statistics in post-order traversals, and doing $O(1)$ query math using prefix/suffix bounding arrays.

1. Rerooting & Tree DP

834. Sum of Distances in Tree

Idea: Classic Rerooting DP. First pass (post-order) calculates the `count` of nodes in each subtree and the `res` (sum of distances) for the root θ . Second pass (pre-order) shifts the root dynamically: `res[child] = res[parent] - count[child] + (N - count[child])`.

```

int[] res, count;
List<Set<Integer>> graph;

public int[] sumOfDistancesInTree(int n, int[][] edges) {
    res = new int[n];
    count = new int[n];
    graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new HashSet<>());
    for (int[] e : edges) {
        graph.get(e[0]).add(e[1]);
        graph.get(e[1]).add(e[0]);
    }

    dfs(0, -1);      // Pass 1: compute count[] and res[] for root 0
    dfs2(0, -1, n);  // Pass 2: Reroot and shift the sum formula down
    return res;
}

private void dfs(int node, int parent) {
    count[node] = 1;
    for (int child : graph.get(node)) {
        if (child != parent) {
            dfs(child, node);
            count[node] += count[child];
            res[node] += res[child] + count[child];
        }
    }
}

private void dfs2(int node, int parent, int n) {
    for (int child : graph.get(node)) {
        if (child != parent) {
            // When moving root from 'node' to 'child':
            // 'child' gets CLOSER to (n - count[child]) nodes.
            // 'child' gets FURTHER from count[child] nodes.
            res[child] = res[node] - count[child] + (n - count[child]);
            dfs2(child, node, n);
        }
    }
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

1245. Tree Diameter

Idea: Unrooted tree. Pick an arbitrary node (0), run BFS to find the furthest node A . Run BFS from A to find the furthest node B . The distance from A to B is the tree's true diameter.

```

public int treeDiameter(int[][] edges) {
    int n = edges.length + 1;
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    for (int[] e : edges) {
        graph.get(e[0]).add(e[1]);
        graph.get(e[1]).add(e[0]);
    }

    int[] firstPass = bfs(0, n, graph);    // Find furthest node from arbitrarily chosen 0
    int[] secondPass = bfs(firstPass[0], n, graph); // Find furthest node from that node
    return secondPass[1]; // Distance is the diameter
}

// Returns [furthestNode, distance]
private int[] bfs(int start, int n, List<List<Integer>> graph) {
    Queue<Integer> q = new LinkedList<>();
    boolean[] visited = new boolean[n];
    q.offer(start);
    visited[start] = true;

    int lastNode = start, dist = -1;
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            lastNode = q.poll();
            for (int neighbor : graph.get(lastNode)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.offer(neighbor);
                }
            }
        }
        dist++;
    }
    return new int[]{lastNode, dist};
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

2. DP & Passing Counting Arrays

1519. Number of Nodes in Subtree with Same Label

Idea: Post-order traversal. Each node returns a `int[26]` frequency array up to its parent. The parent merges the children arrays into its own and stores the result for its specific label.

```

int[] ans;

public int[] countSubTrees(int n, int[][] edges, String labels) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    for (int[] e : edges) {
        graph.get(e[0]).add(e[1]);
        graph.get(e[1]).add(e[0]);
    }
    ans = new int[n];
    dfs(0, -1, graph, labels.toCharArray());
    return ans;
}

private int[] dfs(int node, int parent, List<List<Integer>> graph, char[] labels) {
    int[] count = new int[26];
    count[labels[node] - 'a'] = 1; // Count itself

    for (int child : graph.get(node)) {
        if (child != parent) {
            int[] childCount = dfs(child, node, graph, labels);
            for (int i = 0; i < 26; i++) { // Merge
                count[i] += childCount[i];
            }
        }
    }
    ans[node] = count[labels[node] - 'a']; // Record answer for this subtree
    return count;
}

```

- **Complexity:** Time $O(26 \times N) \approx O(N)$, Space $O(N)$

2246. Longest Path With Different Adjacent Characters

Idea: Post-order DP. Track the two longest paths coming from children where `child.char != root.char`. The longest path THRU the root is `longest1 + longest2 + 1`.

```

int maxPath = 1;
public int longestPath(int[] parent, String s) {
    int n = parent.length;
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    for (int i = 1; i < n; i++) {
        graph.get(parent[i]).add(i); // Directed tree from root down
    }

    dfs(0, graph, s.toCharArray());
    return maxPath;
}

private int dfs(int node, List<List<Integer>> graph, char[] s) {
    int longest1 = 0, longest2 = 0;

    for (int child : graph.get(node)) {
        int childLen = dfs(child, graph, s);
        if (s[node] == s[child]) continue; // Broken path requirement

        if (childLen > longest1) {
            longest2 = longest1;
            longest1 = childLen;
        } else if (childLen > longest2) {
            longest2 = childLen;
        }
    }

    maxPath = Math.max(maxPath, longest1 + longest2 + 1); // Bridge
    return longest1 + 1; // Return the single longest extensible branch
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

3. Querying & Indexing Complexities

2385. Amount of Time for Binary Tree to Be Infected

Idea: Identical trick to "Distance K" (863). Convert into an Undirected Graph (Adjacency List) by traversing once, then BFS heavily outward from the `start` node. The time to infect the whole tree is the maximum depth of that BFS queue minus 1.

```

Map<Integer, List<Integer>> graph = new HashMap<>();

public int amountOfTime(TreeNode root, int start) {
    buildGraph(root, null); // Turns tree into an undirected graph

    Queue<Integer> q = new LinkedList<>();
    Set<Integer> visited = new HashSet<>();
    q.offer(start);
    visited.add(start);

    int time = -1;
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            int curr = q.poll();
            for (int neighbor : graph.getOrDefault(curr, new ArrayList<>())) {
                if (visited.add(neighbor)) {
                    q.offer(neighbor);
                }
            }
        }
        time++; // One outward pulse equals one minute
    }
    return time;
}

private void buildGraph(TreeNode node, TreeNode parent) {
    if (node == null) return;
    graph.putIfAbsent(node.val, new ArrayList<>());
    if (parent != null) {
        graph.get(node.val).add(parent.val);
        graph.get(parent.val).add(node.val);
    }
    buildGraph(node.left, node);
    buildGraph(node.right, node);
}

```

- **Complexity:** Time $O(N)$, Space $O(N)$

2458. Height of Binary Tree After Subtree Removal Queries

Idea: Track the tree's height via Pre-order (left-to-right) and Post-order (right-to-left). $map[node.val] = \max(heightBeforeNodeLtoR, heightBeforeNodeRtoL)$. It allows $O(1)$ response per query!


```

int currentMaxHeight = 0;
Map<Integer, Integer> res = new HashMap<>();

public int[] treeQueries(TreeNode root, int[] queries) {
    // Left-to-right traversal
    currentMaxHeight = 0;
    traverseLeftToRight(root, 0);

    // Right-to-left traversal
    currentMaxHeight = 0;
    traverseRightToLeft(root, 0);

    int[] ans = new int[queries.length];
    for (int i = 0; i < queries.length; i++) {
        ans[i] = res.get(queries[i]); // O(1) query time!
    }
    return ans;
}

private void traverseLeftToRight(TreeNode node, int depth) {
    if (node == null) return;
    res.put(node.val, currentMaxHeight); // Record max height seen BEFORE entering this subtree
    currentMaxHeight = Math.max(currentMaxHeight, depth);
    traverseLeftToRight(node.left, depth + 1);
    traverseLeftToRight(node.right, depth + 1);
}

private void traverseRightToLeft(TreeNode node, int depth) {
    if (node == null) return;
    // Overwrite the map with the max of (Left-To-Right Max, Right-To-Left Max)
    res.put(node.val, Math.max(res.get(node.val), currentMaxHeight));
    currentMaxHeight = Math.max(currentMaxHeight, depth);
    traverseRightToLeft(node.right, depth + 1); // Note: traversing right first!
    traverseRightToLeft(node.left, depth + 1);
}

```

- **Complexity:** Time $O(N + Q)$, Space $O(N)$

2689. Extract Kth Character From The Rope Tree

Idea: Binary Tree Search. Because each node stores bounds/lengths, if $k > \text{left_subtree_length}$, you subtract the left length and search right. If k is smaller, you plunge left. (Standard Rope string boundary descent).

```

// Definition for a RopeTreeNode (as specified by problem):
// class RopeTreeNode {
//     int len;
//     String val;
//     RopeTreeNode left;
//     RopeTreeNode right;
// }

public char getKthCharacter(RopeTreeNode root, int k) {
    if (root.len == 0) return root.val.charAt(k - 1); // Hit a true leaf!

    int leftLength = getLength(root.left);
    if (k <= leftLength) {
        return getKthCharacter(root.left, k);
    } else {
        return getKthCharacter(root.right, k - leftLength);
    }
}

private int getLength(RopeTreeNode node) {
    if (node == null) return 0;
    if (node.len > 0) return node.len;
    return node.val.length();
}

```

- **Complexity:** Time $O(H)$, Space $O(H)$ for recursive call stack